Università degli Studi di Firenze
Dipartimento di Ingegneria dell'Informazione

MBPConfigurator

Configuratore di PC

Autori:
Tommaso Lapenna
Lucas Melo Alves
Samuele Baldi

N° matricola:
7112937
7777777
7116707

Corso:
Ingegneria del Software

Docente del corso:
Prof. Enrico Vicario

Anno Accademico 2025–2026

Indice

1 Introduzione **2**
1.1 Statement . . . 2
1.2 Obiettivo e Requisiti . . . 2
1.3 Tecnologie e Strumenti Utilizzati . . . 2

2 Progettazione **3**
2.1 Architettura e Design . . . 3
2.2 Use Case Diagram . . . 4
2.3 Use Case Template . . . 4
2.4 Mock-Ups . . . 9
2.5 Navigation Diagram . . . 9
2.6 Class Diagram . . . 10
2.7 ER Diagram e Modello Relazionale . . . 10

3 Implementazione **10**
3.1 Domain . . . 10
3.1.1 Model . . . 11
3.1.2 Event . . . 15
3.1.3 Rule . . . 15
3.2 Application . . . 17
3.3 ORM . . . 17

4 Design Pattern e Strutture Dati **18**
4.1 Specification . . . 18
4.1.1 Creazione . . . 22
4.1.2 Logica . . . 25
4.1.3 Composizione con Guardie . . . 26
4.2 Observer . . . 28
4.3 Memento . . . 29
4.4 Facade . . . 30
4.5 Altri Design Pattern . . . 31
4.6 Strutture Dati Utilizzate . . . 31

1 Introduzione

1.1 Statement

Il progetto **MPBConfigurator** modella un'applicazione che permette all'utente di creare una pc build, confrontare i componenti e verificarne la compatibilità ed il prezzo totale.

L'interfaccia del configuratore permette di inserire via via i componenti scelti, annullare le scelte effettuate o ripristinare l'annullamento. Inoltre sono visibili informazioni su cosa modificare e/o inserire per ottenere un pc funzionante tramite il sistema di compatibilità in tempo reale.

Tutto il catalogo dei componenti è disponibile all'utente tramite l'apposito database, quest'ultimo è gestito dall'operatore del sistema.

1.2 Obiettivo e Requisiti

???? maybe?

1.3 Tecnologie e Strumenti Utilizzati

Java 25	Implementazione dell'applicazione
IntelliJ	IDE
JUnit 5	Testing dell'applicazione
SQLite	Database dell'applicazione
PlantUML	Strumento di creazione dei diagrammi UML
Github	Versionamento e condivisione del codice

Tabella 1: Tecnologie utilizzate

Sono inoltre stati utilizzati i seguenti modelli per aiutare lo sviluppo del codice e la progettazione:

- Claude Sonnet 5.5
- Claude Agent
- Gemini Pro 3.1, Flash 3.5
- LLM locali: (Gemma 4 e4b, Qwen 3.5 9b, Nemotron 3-nano 4b)
- **GUI** rappresenta l'UI layer.
- **Application** rappresenta l'Application Layer, e si occuppa di gestire le richieste dell'utente e la coordinazioni tra i vari elementi (in modo da "risolvere" gli use cases).

- **Domain** rappresenta il Domain Layer: contiene il modello di tutte le entità del dominio in questione (tutti i tipi di componenti e le build), insieme a tutta la logica e regole che le gestiscono.
- **ORM** rappresenta il Persisistence Layer, definisce le repository interfaces deinite dal dominio.

2 Progettazione

2.1 Architettura e Design

L'applicazione è strutturata secondo un'architettura a livelli, inspirata al Domain Driven Desgin

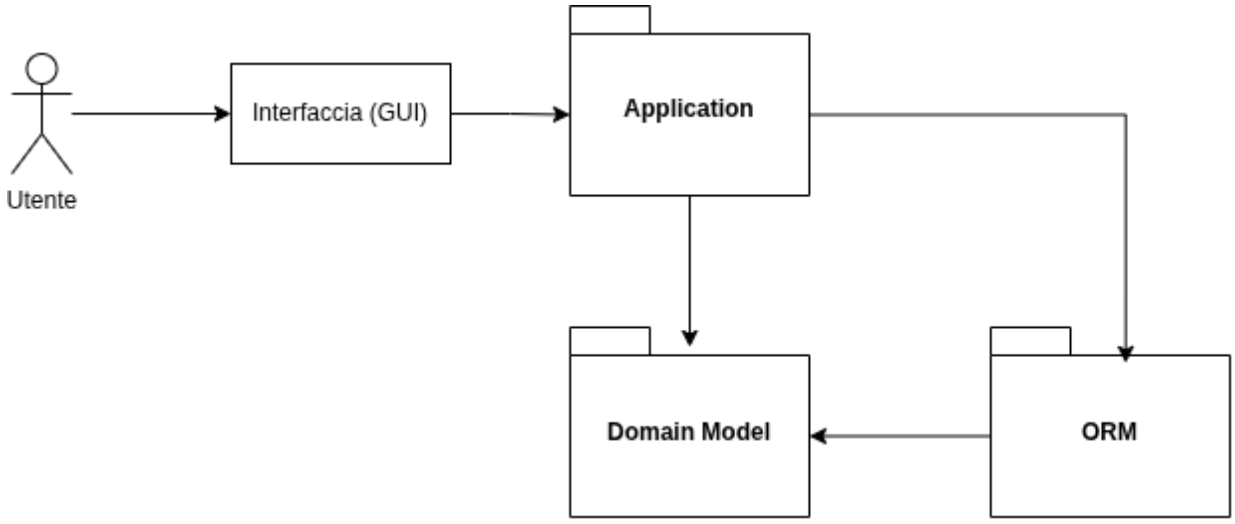


Figura 1: Package Diagram

NEL DIAGRAMMA MANCA IL DB
DEFINIZIONE LAYER

2.2 Use Case Diagram

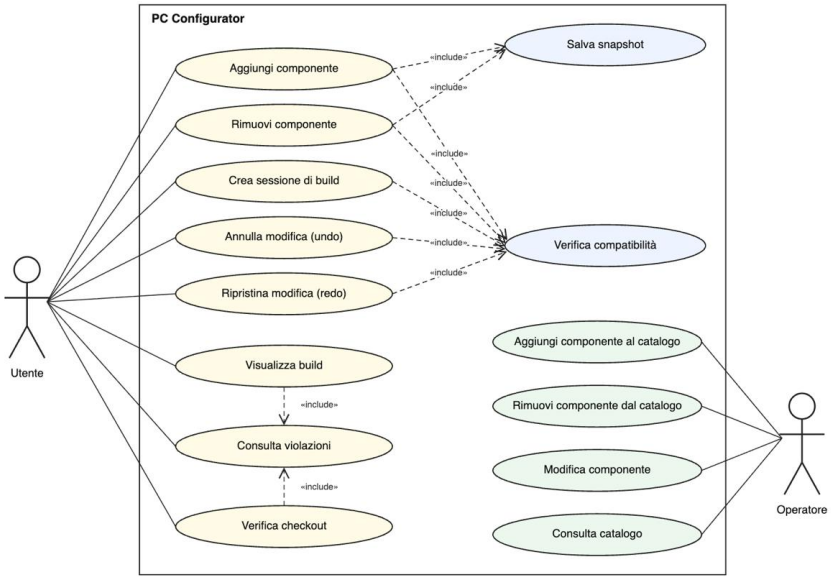


Figura 2: Use Case Diagram

2.3 Use Case Template

UC1 — Crea sessione di build	
Descrizione	L'utente avvia una nuova sessione di configurazione PC. Il sistema crea una `PcBuildSession` vuota con `PcBuilder` e `SnapshotHistory` inizializzati.
Livello	User goal
Attori	Utente
Pre-condizioni	L'utente è autenticato nel sistema.
Flusso base	1. L'utente seleziona «Nuova build». 2. Il sistema crea una `PcBuildSession` con `PcBuilder` vuoto. 3. Il sistema inizializza la `SnapshotHistory`. 4. Il sistema mostra l'interfaccia di configurazione con tutti gli slot vuoti.
Flussi alternativi	1a. L'utente sceglie di caricare una build salvata: il sistema ripristina la sessione precedente (→ UC6).
Post-condizioni	Una sessione di build attiva è disponibile.
Test	

UC2 — Aggiungi componente	
Descrizione	L'utente aggiunge un componente dal catalogo alla build corrente. Il sistema salva uno snapshot e verifica la compatibilità automaticamente.
Livello	User goal
Attori	Utente
Pre-condizioni	Una sessione di build è attiva. Il catalogo contiene almeno un componente.
Flusso base	1. L'utente esplora il catalogo e seleziona un componente. 2. L'utente clicca «Aggiungi alla build». 3. Il sistema aggiunge il componente tramite `PcBuildSession.add()`. 4. Il sistema salva uno snapshot nella `SnapshotHistory` «include UC-Snapshot». 5. Il sistema esegue la verifica di compatibilità «include UC-Verifica». 6. Il sistema aggiorna la vista della build e mostra eventuali violazioni.
Flussi alternativi	3a. La cardinalità massima per quel tipo è già raggiunta: il sistema mostra un errore e non aggiunge il componente. 5a. La verifica rileva violazioni: il sistema le elenca senza bloccare l'aggiunta.
Post-condizioni	Il componente è nella build. Uno snapshot è salvato. Le violazioni sono aggiornate.
Test	

UC3 — Rimuovi componente	
Descrizione	L'utente rimuove un componente dalla build corrente. Il sistema salva uno snapshot e aggiorna la verifica di compatibilità.
Livello	User goal
Attori	Utente
Pre-condizioni	Una sessione di build è attiva. La build contiene almeno un componente.
Flusso base	1. L'utente seleziona un componente nella build corrente. 2. L'utente clicca «Rimuovi». 3. Il sistema rimuove il componente tramite `PcBuildSession.remove()`. 4. Il sistema salva uno snapshot «include UC-Snapshot». 5. Il sistema esegue la verifica di compatibilità «include UC-Verifica». 6. Il sistema aggiorna la vista della build.
Flussi alternativi	3a. Il componente non è presente nella build: il sistema mostra un errore.
Post-condizioni	Il componente non è più nella build. Snapshot salvato. Violazioni aggiornate.
Test	

UC4 — Annulla modifica (undo)	
Descrizione	L'utente annulla l'ultima modifica apportata alla build, ripristinando lo stato precedente tramite `SnapshotHistory`.
Livello	User goal
Attori	Utente
Pre-condizioni	Una sessione di build è attiva. La `SnapshotHistory` contiene almeno uno snapshot precedente.
Flusso base	1. L'utente clicca «Annulla». 2. Il sistema ripristina lo stato precedente tramite `PcBuildSession.undo()`. 3. Il sistema esegue la verifica di compatibilità sullo stato ripristinato «include UC-Verifica». 4. Il sistema aggiorna la vista della build.
Flussi alternativi	2a. Non esistono snapshot precedenti: il sistema disabilita il pulsante «Annulla» e non esegue alcuna azione.
Post-condizioni	La build è ripristinata allo stato precedente. Le violazioni sono aggiornate.
Test	

UC5 — Ripristina modifica (redo)	
Descrizione	L'utente ripristina una modifica precedentemente annullata tramite la `SnapshotHistory`.
Livello	User goal
Attori	Utente
Pre-condizioni	Una sessione di build è attiva. È stata eseguita almeno un'operazione di undo.
Flusso base	1. L'utente clicca «Ripristina». 2. Il sistema recupera lo stato successivo tramite `PcBuildSession.redo()`. 3. Il sistema esegue la verifica di compatibilità «include UC-Verifica». 4. Il sistema aggiorna la vista della build.
Flussi alternativi	2a. Non esistono snapshot successivi: il sistema disabilita il pulsante «Ripristina».
Post-condizioni	La build è ripristinata allo stato successivo. Le violazioni sono aggiornate.
Test	

UC6 — Visualizza build	
Descrizione	L'utente visualizza la configurazione corrente della build con tutti i componenti selezionati, le violazioni di compatibilità e il wattaggio totale.
Livello	User goal
Attori	Utente
Pre-condizioni	Una sessione di build è attiva.
Flusso base	1. Il sistema mostra i componenti presenti tramite `BuildView` (`getBuild()`). 2. Il sistema mostra le violazioni di compatibilità «include UC7». 3. Il sistema mostra il wattaggio totale (`totalWattage()`).
Flussi alternativi	1a. La build è vuota: il sistema mostra gli slot vuoti per ogni `ComponentType`.
Post-condizioni	L'utente ha una vista aggiornata della build corrente.
Test	

UC7 — Consulta violazioni	
Descrizione	Il sistema mostra la lista delle regole di compatibilità violate nella build corrente, calcolate dal `CompatibilityChecker` tramite le 40 `BuildRule` di `DefaultRules`.
Livello	Subfunction
Attori	Utente (indiretto, tramite UC6)
Pre-condizioni	Una sessione di build è attiva. La verifica di compatibilità è stata eseguita.
Flusso base	1. Il sistema recupera le violazioni tramite `PcBuildSession.getViolations()`. 2. Il sistema mostra la lista con la descrizione di ogni regola violata. 3. Se non ci sono violazioni, mostra «Nessun problema rilevato».
Flussi alternativi	(nessuno)
Post-condizioni	L'utente è informato delle incompatibilità presenti nella build.
Test	

UC8 — Visualizza prezzo	
Descrizione	Il sistema mostra il prezzo di mercato suggerito della build corrente, calcolato come somma dei prezzi indicativi dei componenti. Il prezzo non è vincolante.
Livello	User goal
Attori	Utente
Pre-condizioni	Una sessione di build è attiva.
Flusso base	1. Il sistema somma i prezzi di tutti i componenti nella build. 2. Il sistema mostra il totale con la dicitura «Prezzo di mercato suggerito». 3. Il sistema mostra il disclaimer: «I prezzi sono indicativi e soggetti a variazione».
Flussi alternativi	1a. La build è vuota: il sistema mostra €0,00.
Post-condizioni	L'utente conosce la stima del prezzo della configurazione.
Test	

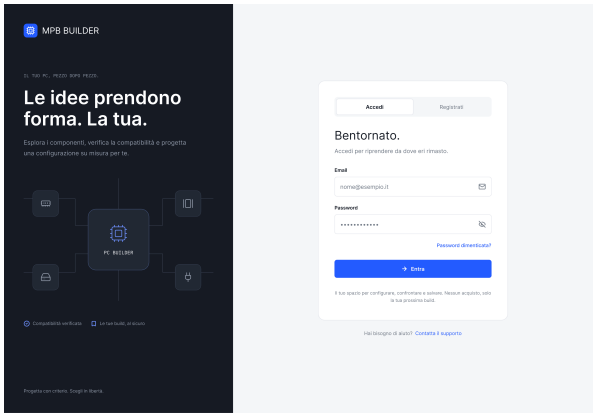
UC9 — Aggiungi al catalogo	
Descrizione	L'operatore aggiunge un nuovo componente al catalogo del sistema, specificandone tutte le caratteristiche tecniche.
Livello	User goal
Attori	Operatore
Pre-condizioni	L'operatore è autenticato. Il componente non è già presente nel catalogo.
Flusso base	1. L'operatore seleziona «Aggiungi componente» e sceglie il tipo. 2. Il sistema mostra il form con i campi richiesti (socket, form factor, ecc.). 3. L'operatore compila i campi e conferma. 4. Il sistema valida i dati inseriti. 5. Il sistema aggiunge il componente al catalogo. 6. Il sistema mostra la conferma dell'operazione.
Flussi alternativi	4a. I dati non sono validi (campo obbligatorio mancante): il sistema segnala l'errore e non salva. 5a. Un componente identico esiste già: il sistema segnala il duplicato.
Post-condizioni	Il componente è disponibile nel catalogo per tutti gli utenti.
Test	

UC10 — Rimuovi dal catalogo	
Descrizione	L'operatore rimuove un componente esistente dal catalogo.
Livello	User goal
Attori	Operatore
Pre-condizioni	L'operatore è autenticato. Il componente esiste nel catalogo.
Flusso base	1. L'operatore cerca e seleziona il componente nel catalogo. 2. L'operatore seleziona «Rimuovi». 3. Il sistema chiede conferma. 4. L'operatore conferma. 5. Il sistema rimuove il componente dal catalogo.
Flussi alternativi	4a. L'operatore annulla: il sistema non esegue alcuna modifica.
Post-condizioni	Il componente non è più disponibile nel catalogo.
Test	

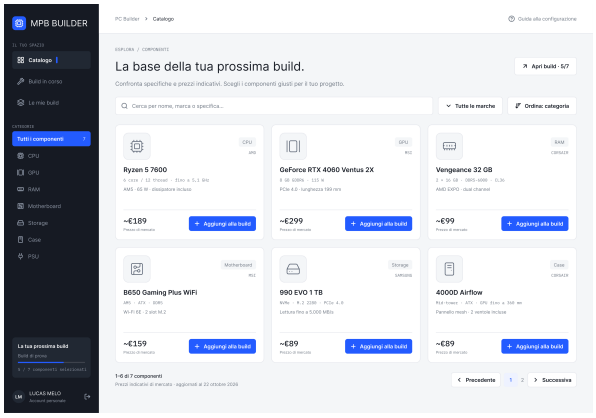
UC11 — Modifica componente nel catalogo	
Descrizione	L'operatore modifica le informazioni di un componente esistente nel catalogo.
Livello	User goal
Attori	Operatore
Pre-condizioni	L'operatore è autenticato. Il componente esiste nel catalogo.
Flusso base	1. L'operatore cerca e seleziona il componente. 2. Il sistema mostra il form con i dati attuali. 3. L'operatore modifica i campi desiderati e conferma. 4. Il sistema valida i nuovi dati. 5. Il sistema aggiorna il componente nel catalogo.
Flussi alternativi	4a. I dati modificati non sono validi: il sistema segnala l'errore e non salva.
Post-condizioni	Il componente nel catalogo riflette i nuovi dati.
Test	

UC12 — Consulta catalogo	
Descrizione	L'operatore visualizza la lista completa dei componenti nel catalogo, con possibilità di filtrare per tipo.
Livello	User goal
Attori	Operatore
Pre-condizioni	L'operatore è autenticato.
Flusso base	1. L'operatore accede alla sezione catalogo. 2. Il sistema mostra tutti i componenti suddivisi per tipo (CPU, GPU, RAM, ...). 3. L'operatore può filtrare per tipo, marca o prezzo.
Flussi alternativi	2a. Il catalogo è vuoto: il sistema mostra «Nessun componente presente».
Post-condizioni	L'operatore ha una vista aggiornata del catalogo.
Test	

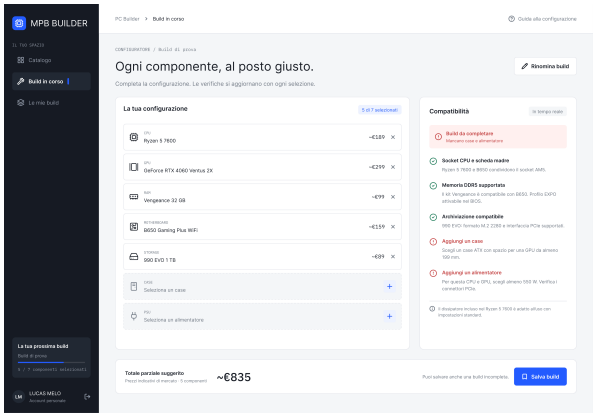
2.4 Mock-Ups



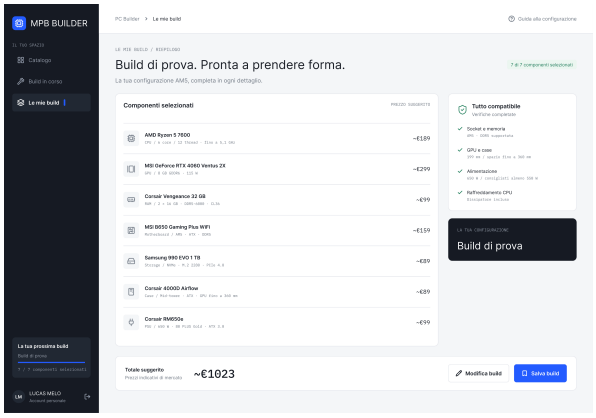
(a) Mockup 1



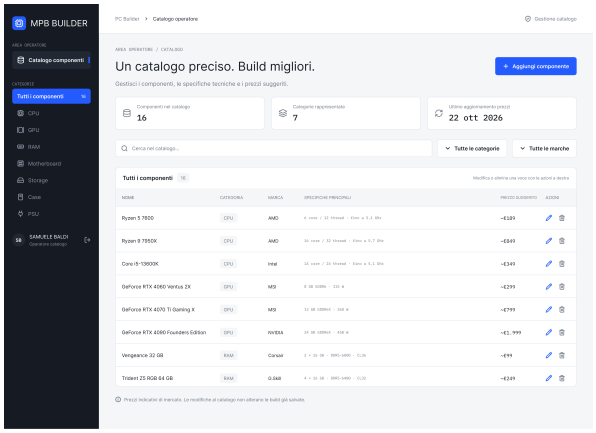
(b) Mockup 2



(c) Mockup 3



(d) Mockup 4



(e) Mockup 5

Figura 3: Mockup dell'interfaccia

2.5 Navigation Diagram

da fare

2.6 Class Diagram

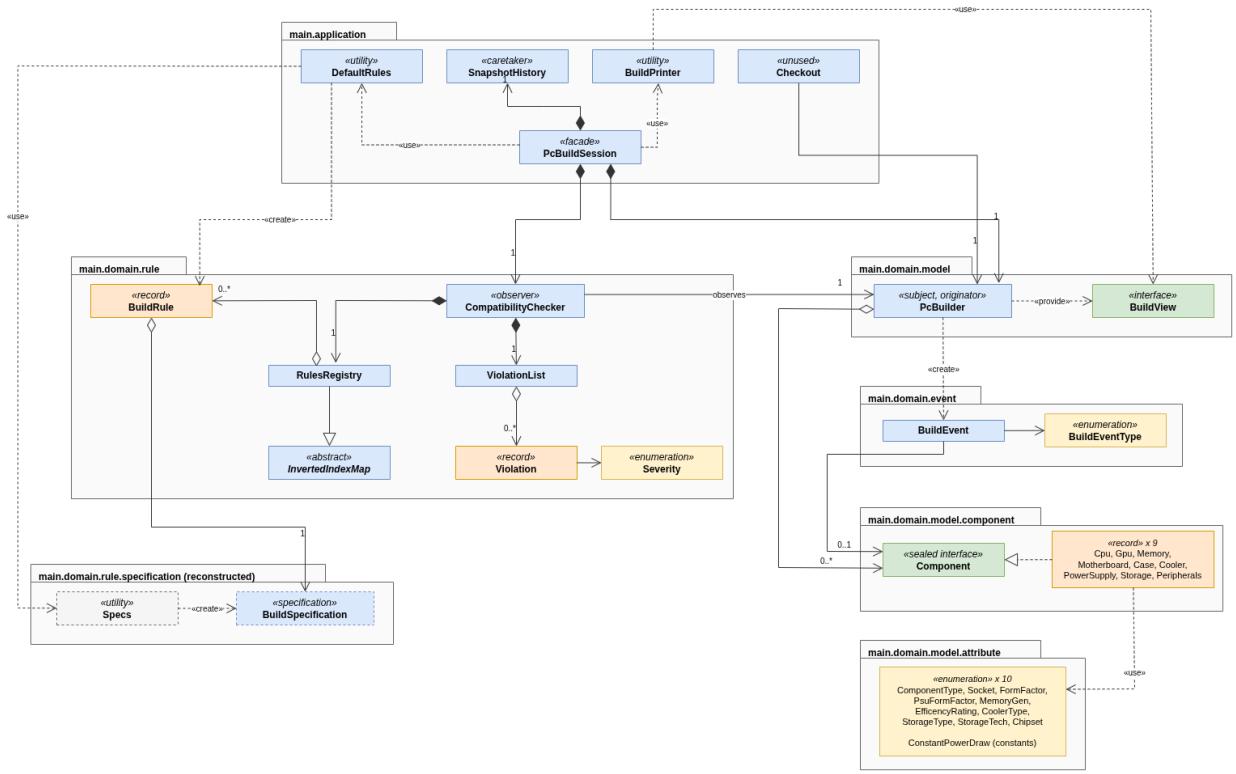


Figura 4: Enter Caption

2.7 ER Diagram e Modello Relazionale

deve farla il baldè

3 Implementazione

3.1 Domain

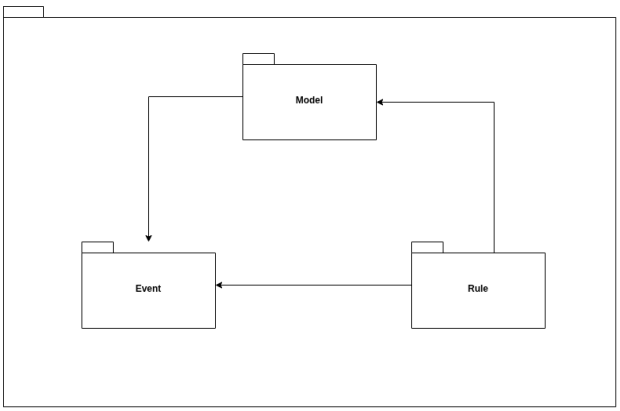


Figura 5: Domain Package Diagram

Il dominio contiene sia il modello che le sue business rule, nei corrispettivi pacchetti *.model* e *rule*

3.1.1 Model

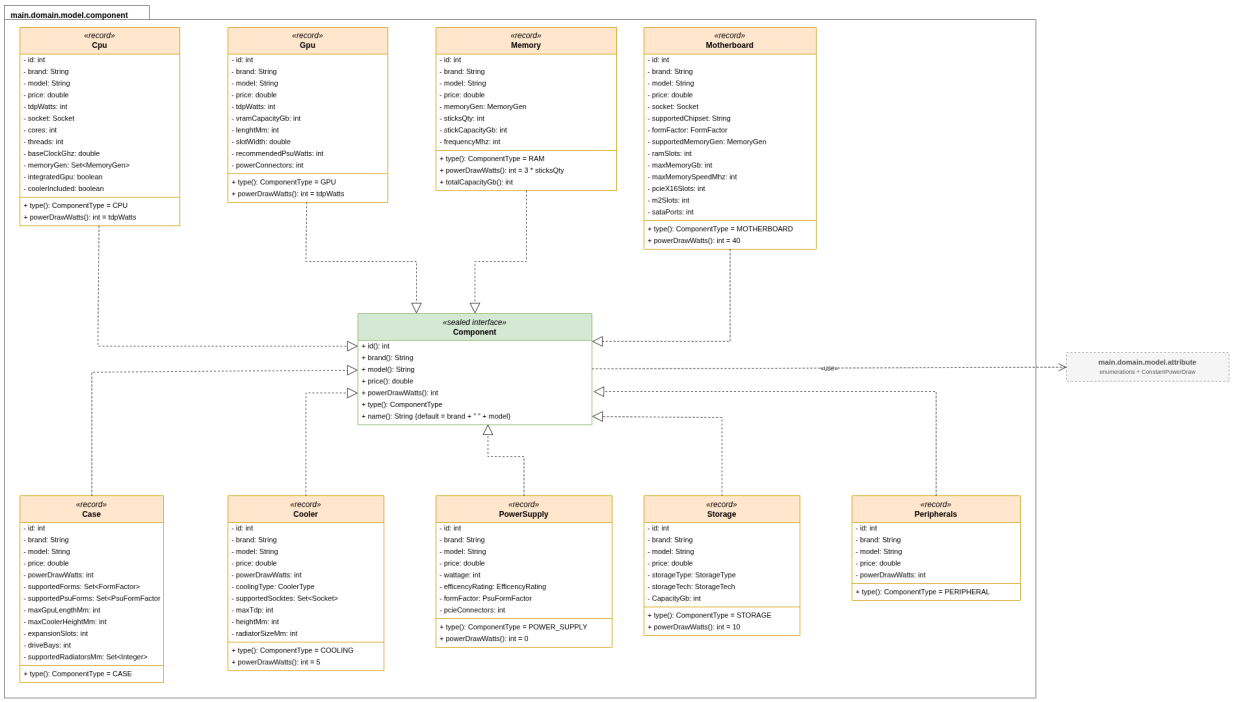


Figura 6: Enter Caption

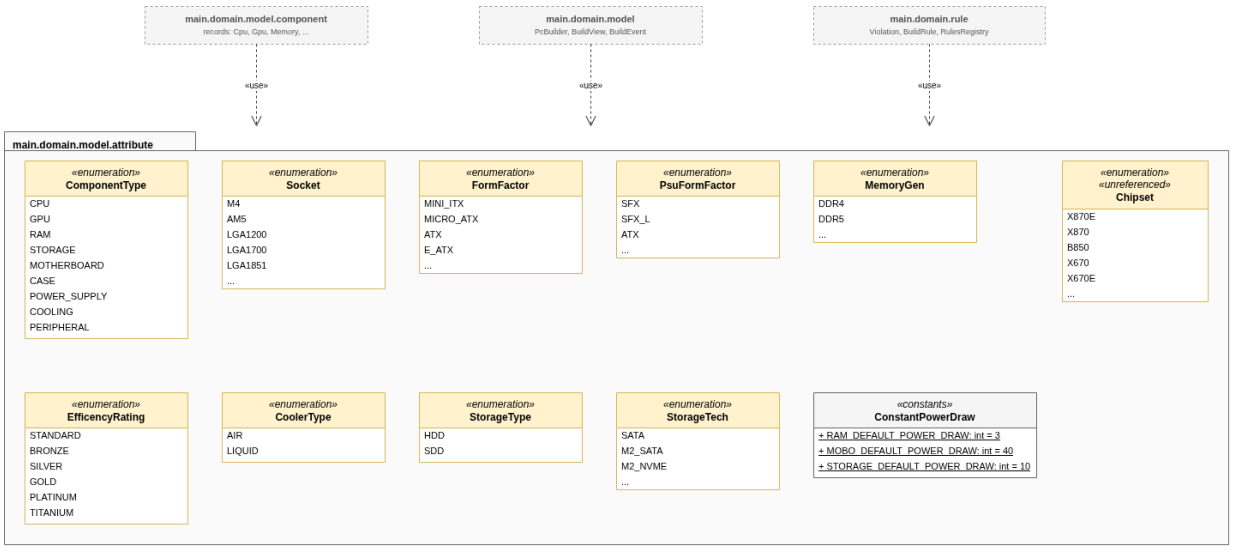


Figura 7: Enter Caption

Il modello è composto da due sottopacchetti:

- **component** contiene l'interfaccia chiusa *Component*, che viene implementata dalle tipologie possibili tipologie di componenti come *record*. La scelta di implementarli

come record deriva dalla loro natura costante e immutabile della collezione di informazioni che rappresenta ogni componente, perciò è possibile sfruttare la sintassi più semplice e i vantaggi per la jvm.

```java
//Component.java
package main.domain.model.component;

public sealed interface Component
        permits Case, Gpu, Peripherals, Cooler, Cpu,
        Memory, Motherboard, PowerSupply, Storage {

    int id();
    String brand();
    String model();
    double price();

    int powerDrawWatts();

    public ComponentType type();
    default String name() { return brand() + " " + model(); }

}
```

Listing 1: A simple Java class

I record in questione sono i seguenti:

  - `Cpu`
  - `Gpu`
  - `Motherboard`
  - `Memory`
  - `Storage`
  - `Case`
  - `PowerSupply`
  - `Peripherals`

- **attribute** contiene invece le enumerazioni che determinano le tipologie le caratteristiche dei componenti. queste sono:
  - `ComponentType`
  - `Chipset`
  - `CoolerType`

- `EfficencyRating`
- `FormFactor`
- `MemoryGen`
- `PsuFormFactor`
- `Socket`
- `StorageTech`
- `StorageType`
- `ConstantPowerDraw` (non è un'enum a differenza degli altri)

```java
//ComponentType.java
package main.domain.model.attribute;

public enum ComponentType {
    CPU,
    GPU,
    RAM,
    STORAGE,
    MOTHERBOARD,
    CASE,
    POWER_SUPPLY,
    COOLING,
    PERIPHERAL
}
```

Listing 2: A simple Java class

Tutti questi componenti verranno poi composti tra di loro per ottenere la build completa del pc, che rappresenta il punto centrale dell'applicazione. La classe che si occupa di questo è `PcBuilder`, che raccoglie tutti i componenti in un'*EnumMap*: le chiavi sono quelle fornite dall'enumerazione `ComponentType`, mentre i valori sono delle liste di elementi `Component`, che contengono i componenti per a seconda della corrispettiva tipologia.
Le liste possono contenere più elementi per tipologia, questo però è un comportamento consentito dal metodo `addComponent()` e da `CompatibilityChecker`: un utente può inserire ad esempio due cpu alla configurazione (magari per confrontarle in un attimo di decisione), questa irregolarità verrà poi segnalata in `ViolationList` e non sarà possibile effettuare il checkout finché queste persistono.

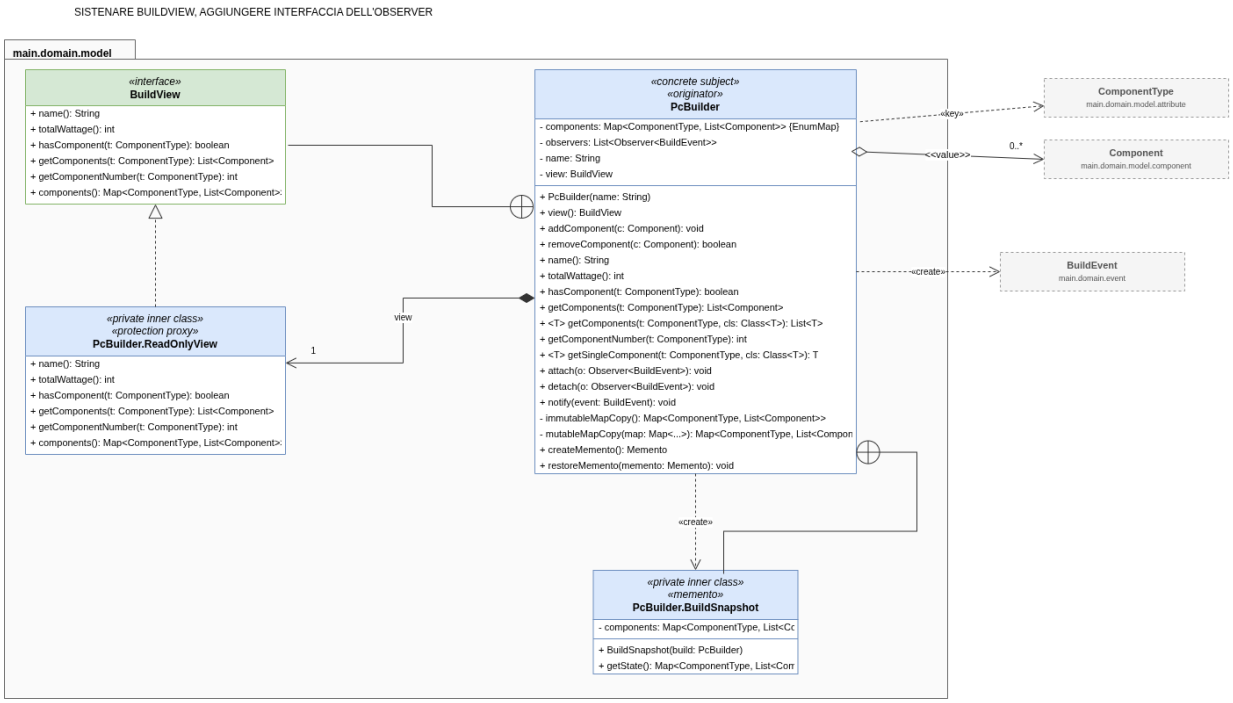


Figura 8: Enter Caption

`PcBuilder` contiene al suo interno due classi astratte:

- `ReadonlyView`: implementa l'interfaccia pubblica BuildView. Se questa interfaccia fosse implementata da PcBuilder direttamente, sarebbe possibile effettuare il cast `(PcBuilder) session.getBuild()...` e violare i limiti di sola lettura imposta dall'interfaccia. Implementare invece quetsa interfaccia in una classe privata che funge da *Proxy*, rende impossibile questo "escamotage".

- `BuildSnapshot`: permette l'implementazione corretta di memento. Questa classe privata è la migliore alternativa alla funzionalità *friend* del C++ (mancante in java) utilizzata in *GoF* per implementare memento: quindi BuildSnapshot implementa l'interfaccia pubblica `Memento` e il tipo che ritorna *createMemento* è il medesimo, così all'esterno non è possibile effettuare un cast alla classe privata i questione preservando la segretezza dello stato del memento al di fuori di *Originator*.

3.1.2 Event

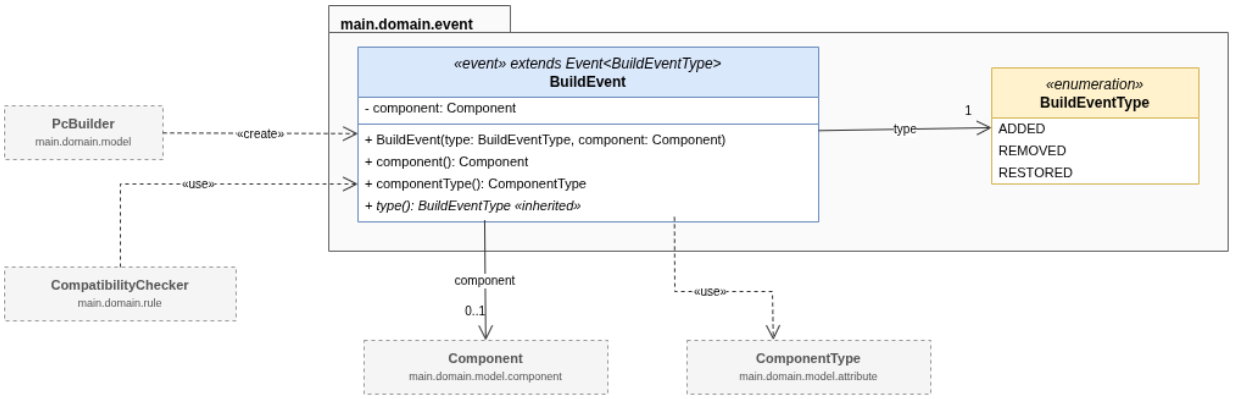


Figura 9: Enter Caption

Contiene le informazioni su che modifica è stata effettuata...

3.1.3 Rule

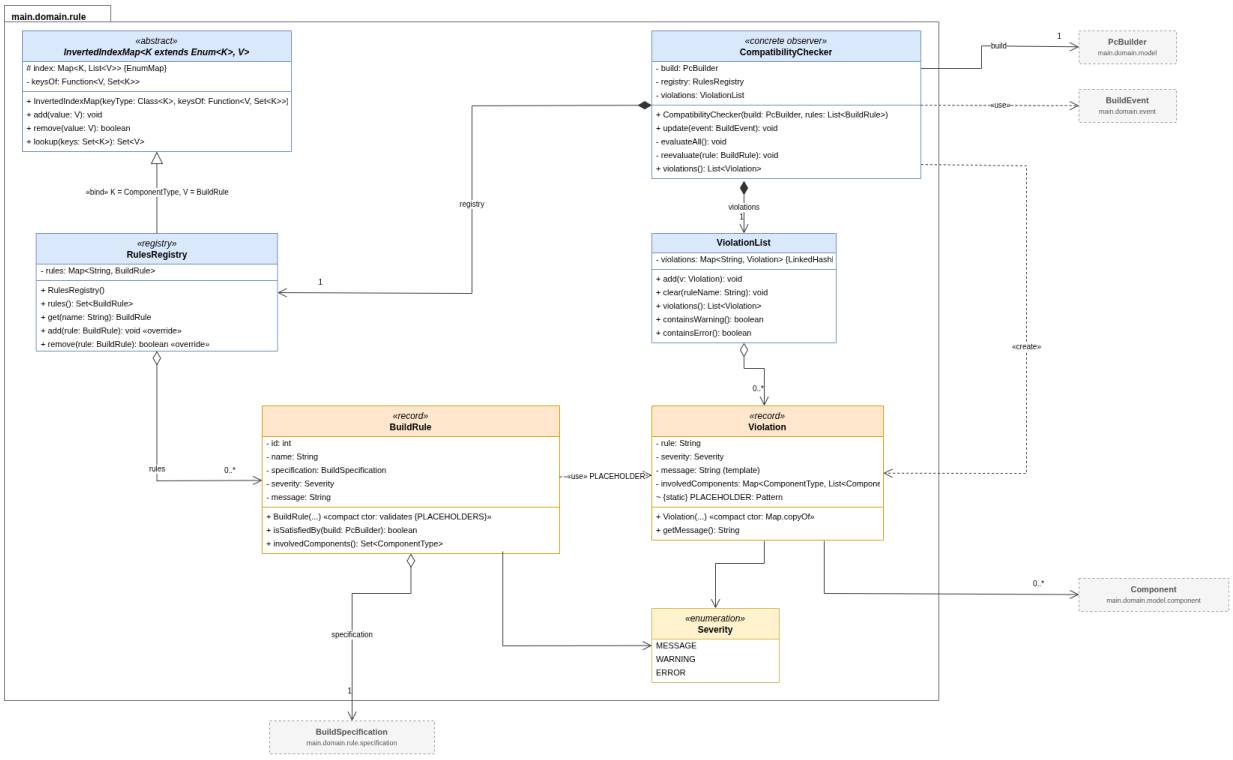


Figura 10: Enter Caption

Il vero cuore di questa applicazione è il sistema di controllo per verificare le compatibilità tra i vari componenti. Ogni componente ha delle specifiche relative alle proprie dimensioni o tecnologie utilizzate (ad esempio le grandezze dei case e quindi cosa può effettivamente contenere o se il socket della cpu corrisponde con quello della scheda madre). Bisogna anche tenere conto se il numero di componenti inseriti è corretto o se manca un componente essenziale per poter utilizzare il pc.

```java
//CompatibilityChecker.java
package main.domain.rule;

import java.util.*;

public class CompatibilityChecker implements Observer<BuildEvent> {
    private final PcBuilder build;
    private final RulesRegistry registry = new RulesRegistry();
    private final ViolationList violations = new ViolationList();

    public CompatibilityChecker(PcBuilder build, List<BuildRule>
        rules) {
        this.build = build;
        build.attach(this);
        // This lambda is the equivalent of r -> registry.add(r), it
            was suggested by the IDE
        rules.forEach(registry::add);
        evaluateAll();
    }

    @Override
    public void update(BuildEvent event) {
        if (event.type() == BuildEventType.RESTORED)
            evaluateAll();
        else {
            for (BuildRule rule : registry.lookup(Set.of(event.
                componentType()))) {
                reevaluate(rule);
            }
        }
    }

    private void evaluateAll() {
        for (BuildRule rule : registry.rules())
            reevaluate(rule);
    }

    private void reevaluate(BuildRule rule) {
        if (!rule.isSatisfiedBy(build)) {
            Map<ComponentType, List<Component>> involved = new
                LinkedHashMap<>();
            for (ComponentType type : rule.involvedComponents()) {
                involved.put(type, List.copyOf(build.getComponents(
                    type)));
            }
            violations.add(new Violation(rule.name(), rule.severity()
                , rule.message(), involved));
```

```
42          } else
43              violations.clear(rule.name());
44      }
45
46      public List<Violation> violations() {
47          return violations.violations();
48      }
49  }
```

Listing 3: A simple Java class

Il motore principale con il quale vengono create le regole di compatibiltà si basa sul design pattern *Specification*: questo permette di definire business rules, con esito di validazione booleano, che possono essere combinate tra loro tramite operatori logici.

3.2 Application

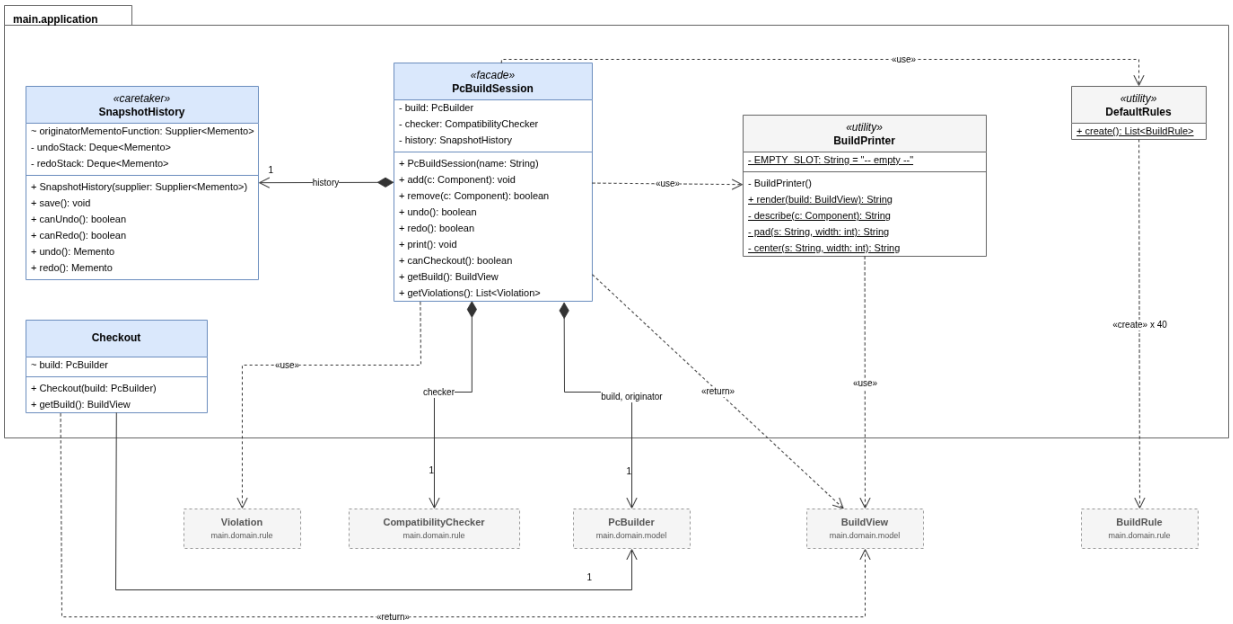


Figura 11: Enter Caption

Il pacchetto `application` contiene

3.3 ORM

TODO

4 Design Pattern e Strutture Dati

4.1 Specification

Specification è un design pattern definito da Eric Ewans in Domain Driven Design (da aggiungere ref).

- **Problema**: "You need to check that only suitable objects are used for a certain role", si parla quindi di responsabilità di validazione.
- **Soluzione**: "Create a specification that is able to tell if a candidate object matches some criteria. The specification has a method isSatisfiedBy(anObject): Boolean that returns "true" if all criteria are met by anObject".

Tra le varie implementazioni proposte, quella scelta è Composite Specification, che permette la massima flessibiltà e la possibilità di combinare le specifiche.

La struttura composita permette quindi di poter usare le operazioni di congiunzione logiche and e or) che compongono la specifica ad albero, con l'implementazione della versione composite di Specification si crea infatti un *interpreter* per la lingua della specifiche.

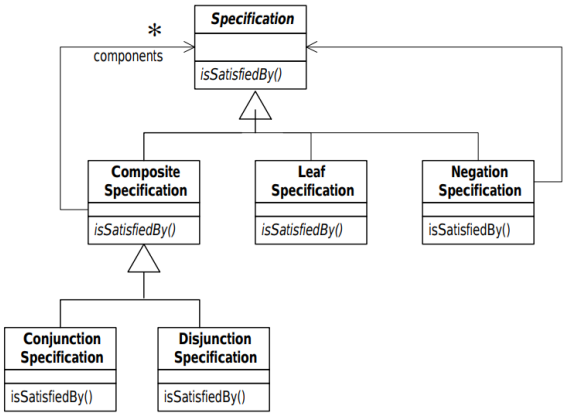


Figura 12: Enter Caption

Grazie quindi alla caratteristica principale di un composite, ovvero quella di avere un interfaccia comune sia per i padri sia per le foglie nell'albero (in questo caso l'interfaccia è Specification), basta verificare la specifica risultante.

Esempio: SpecificationA $\wedge$ ($\neg$SpecificationB $\vee$ SpecificationC)

$\iff$ `SpecificationTot = SpecificationA.and(`

`SpecificationB.not().or(SpecificationC))`

L'intera specifica potrà poi essere verificaza attravaerso

`SpecificationTot.isSatisfiedBy(Object)`

Specification permette quindi ti poter definire degli oggetti per validare la build in base alle specifiche, per poter sfruttarla al massimo però occorre ampliare la sua flessibilità:

1. Nella sua definizione "standard", `isSatisfiedBy()` restituisce solo un booleano, quando nel caso dell'applicazione servono informazioni aggiuntive. Quindi le specifiche sono utilizzate per creare le `BuildRule`, che contengono quest'ultime, un nome, la gravità delle violazioni, i componenti coinvolti e degli eventuali messaggi da dare all'utente.
   Se queste regole sono violate, allora viene create una `Violation`, che contiene le informazioni sulla regola violata e la sua presenza (in caso di gravità `Severity.ERROR`) previene il checkout.

2. Siccome le **BuildRule** hanno bisogno dei componenti che sono coinvolti nella loro specifica, ma l'atto di dover scrivere manualmente la lista `InvolvedComponents` esplicitamente riguardante più componenti è prono a errori. Entra quindi in gioco `BuildSpecification`, che aggiunge il metodo `dependsOn()` che grazie al meccanismo di `combine` è in grado di raccogliere tutte le dipendenze di componenti nell'albero composito della specifica.

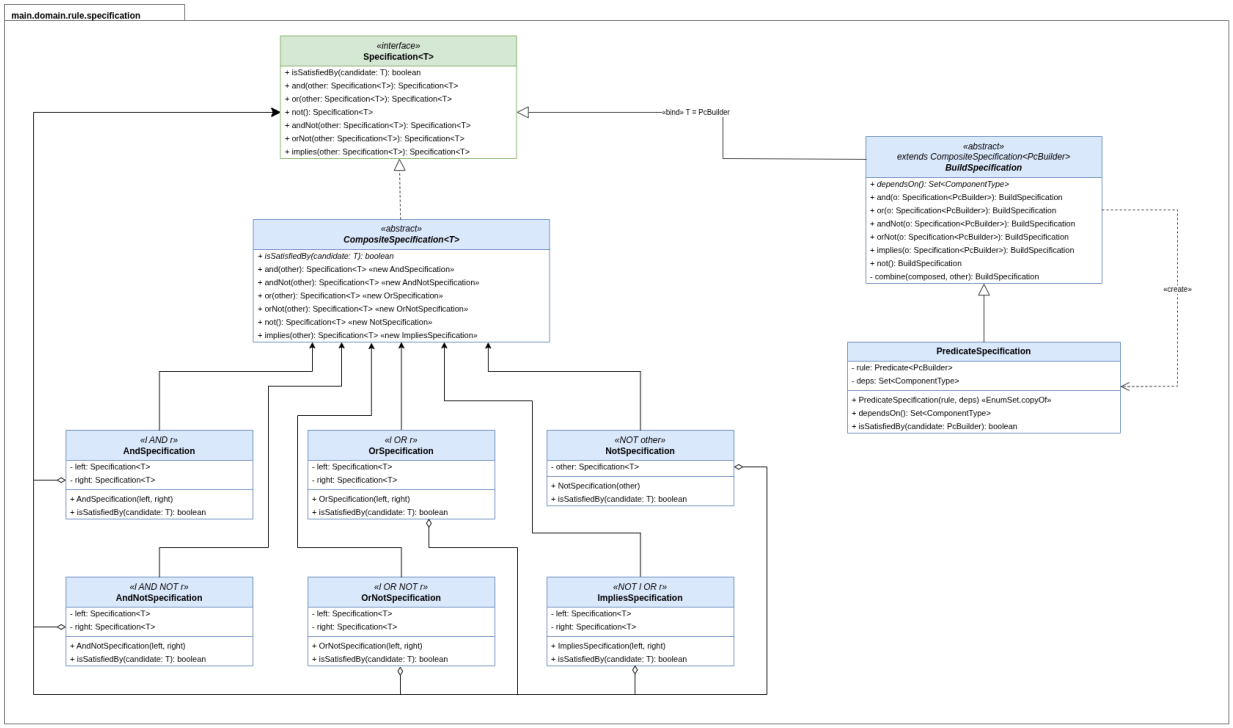


Figura 13: Enter Caption

```java
//Specification.java
package main.pattern.specification;

public interface Specification<T> {
```

```
5       boolean isSatisfiedBy(T candidate);
6
7       Specification<T> and(Specification<T> other);
8       Specification<T> or(Specification<T> other);
9       Specification<T> not();
10      Specification<T> andNot(Specification<T> other);
11      Specification<T> orNot(Specification<T> other);
12
13      Specification<T> implies(Specification<T> other);
14
15  }
```

Listing 4: A simple Java class

```
1  //CompositeSpecification.java
2  package main.pattern.specification;
3
4  public abstract class CompositeSpecification <T> implements
       Specification<T>{
5       @Override public abstract boolean isSatisfiedBy(T candidate);
6       @Override public Specification<T> and(Specification<T> other)
           { return new AndSpecification<>(this, other); }
7       @Override public Specification<T> andNot(Specification<T> other)
           { return new AndNotSpecification<>(this, other); }
8       @Override public Specification<T> or(Specification<T> other)
           { return new OrSpecification<>(this, other); }
9       @Override public Specification<T> orNot(Specification<T> other)
           { return new OrNotSpecification<>(this, other); }
10      @Override public Specification<T> not()
           { return new NotSpecification<>(this); }
11      @Override public Specification<T> implies(Specification<T> other)
            { return new ImpliesSpecification<>(this, other); }
12 }
```

Listing 5: A simple Java class

```
1  //BuildSpecification.java
2  package main.domain.rule.specification;
3
4  public abstract class BuildSpecification extends
       CompositeSpecification<PcBuilder> {
5
6       public abstract Set<ComponentType> dependsOn();
7
8       @Override public BuildSpecification and(Specification<PcBuilder>
           o)      { return combine(super.and(o), o); }
9       @Override public BuildSpecification or(Specification<PcBuilder> o
           )       { return combine(super.or(o), o); }
```

```
10      @Override public BuildSpecification andNot(Specification<
            PcBuilder> o)  { return combine(super.andNot(o), o); }
11      @Override public BuildSpecification orNot(Specification<PcBuilder
            > o)   { return combine(super.orNot(o), o); }
12      @Override public BuildSpecification implies(Specification<
            PcBuilder> o) { return combine(super.implies(o), o); }
13      @Override public BuildSpecification not() {
14          return new PredicateSpecification(super.not()::isSatisfiedBy,
                dependsOn());
15      }
16
17      private BuildSpecification combine(Specification<PcBuilder>
            composed, Specification<PcBuilder> other) {
18          if (!(other instanceof BuildSpecification b))
19              throw new IllegalArgumentException(
20                      "Cannot combine with a spec that does not declare
                            its dependencies: " + other);
21          Set<ComponentType> union = EnumSet.noneOf(ComponentType.class
                );
22          union.addAll(dependsOn());
23          union.addAll(b.dependsOn());
24          return new PredicateSpecification(composed::isSatisfiedBy,
                union);
25      }
26  }
```

Listing 6: A simple Java class

```
1   //BuildRule.java
2   package main.domain.rule;
3
4   public record BuildRule (
5                               int id,
6                               String name,
7                               BuildSpecification specification,
8                               Severity severity,
9                               String message
10      ) {
11
12
13      public BuildRule {
14          Matcher m = Violation.PLACEHOLDER.matcher(message);
15          while (m.find()) {
16              ComponentType type;
17              try {
18                  type = ComponentType.valueOf(m.group(1));
19              } catch (IllegalArgumentException e) {
```

```
20                  throw new IllegalArgumentException("Rule " + name + "
                        : unknown placeholder {" + m.group(1) + "}", e);
21              }
22              if (!specification.dependsOn().contains(type))
23                  throw new IllegalArgumentException("Rule " + name + "
                        : placeholder {" + type + "} is not a component
                        the rule depends on");
24          }
25      }
26
27      public boolean isSatisfiedBy(PcBuilder build) {
28          return specification().isSatisfiedBy(build);
29      }
30
31      public Set<ComponentType> involvedComponents() {
32          return specification.dependsOn();
33      }
34  }
```

Listing 7: A simple Java class

4.1.1 Creazione

Una volta definiti tutti gli aspetti di Specification, bisogna definire le classi che vanno a definire effettivamente i requisiti in `isSatisfiedBy()`. Per evitare però di dover creare troppi file e rendere il progetto molto grande, le specifiche sono definite attraverso due classi "base":

1. **ComponentCountSpecification**: ha come parametri il tipo di componente, il minimo e il massimo, `isSatisfiedBy(PcBuilder build)` quindi confronta la quantità di componenti inseriti nella lista di `type` con quelli forniti. *Questo permette di avere una sola classe per tutte le specifiche che operano su una quantità di una categoria di componente.*

```
1  package main.domain.rule.specification;
2
3  public final class ComponentCountSpecification extends
       BuildSpecification {
4      private final ComponentType type;
5      private final int min, max;
6
7      public ComponentCountSpecification(ComponentType type, int
           min, int max) {
8          this.type = type;
9          this.min = min;
10         this.max = max;
```

```
    }

    @Override
    public boolean isSatisfiedBy(PcBuilder build) {
        int n = build.getComponentNumber(type);
        return n >= min && n <= max;
    }

    @Override public Set<ComponentType> dependsOn() { return
        EnumSet.of(type); }
}
```

Listing 8: A simple Java class

2. **PredicateSpecification**: ha come parametri un *Predicate* e i componenti dai quali dipende. Predicate è un'interfaccia funzionale di java che permette di definire un predicato e poterlo verificare con `.test()`. *Questo permette di avere una sola classe per tutte le specifiche che necessitano di un predicato per essere verificate.*

```
package main.domain.rule.specification;

public class PredicateSpecification extends BuildSpecification {

    private final Predicate<PcBuilder> rule;
    private final Set<ComponentType> deps;

    public PredicateSpecification(Predicate<PcBuilder> rule, Set<
        ComponentType> deps) {
        this.rule = rule;
        this.deps = deps.isEmpty() ? Set.of() : EnumSet.copyOf(
            deps);
    }

    @Override
    public Set<ComponentType> dependsOn() {
        return deps;
    }

    @Override
    public boolean isSatisfiedBy(PcBuilder candidate) {
        return rule.test(candidate);
    }
}
```

Listing 9: A simple Java class

Come si può notare queste due classi di specifiche sono le **uniche non astratte**, quindi si possono creare istanze di specifiche **solo** secondo queste due tipologie e caratteristiche.

L'unico passaggio rimasto è quello di scrivere tutti gli oggetti necessari, che vengono definiti dentro `Specs`:

1. `ComponentCountSpecification` viene utilizzato per creare i metodi<z*factory* `atMost`, `atLeast`, e `excacltyOne` (oltre a `multipleExcactlyOne` che verrà spiegato nella sezione successiva). Questi metodi prendeno come parametro il tipo di componente ed un valore numerico

```
1    //Specs.java
2    public static BuildSpecification atMost(ComponentType t, int
        n)  { return new ComponentCountSpecification(t, 0, n); }
3    public static BuildSpecification atLeast(ComponentType t, int
         n) { return new ComponentCountSpecification(t, n,
        Integer.MAX_VALUE); }
4    public static BuildSpecification exactly(ComponentType t, int
         n) { return new ComponentCountSpecification(t, n, n); }
5
6    public static BuildSpecification mutipleExcatclyOne(
        ComponentType... t) {
7        BuildSpecification res = new ComponentCountSpecification(
            t[0], 1, 1);
8        for (int i = 1; i < t.length; i++)
9            res = res.and(new ComponentCountSpecification(t[i],
                1, 1));
10
11       return res;
12   }
```

Listing 10: A simple Java class

2. Le specification con predicati vengono create come oggetti di classe `PredicateSpecification`, dove il predicato di `isSatisfiedBy()` viene dichiarato direttamente nel costruttore, utilizzando le funzioni lambda (che verranno poi eseguite quandi si andrà a fare `Predicate.test()`.

```
1    //Specs.java
2    public static BuildSpecification socketMatchSpecification =
        new PredicateSpecification(
3            b -> b.getSingleComponent(ComponentType.CPU, Cpu.
                class).socket().equals(b.getSingleComponent(
                ComponentType.MOTHERBOARD, Motherboard.class).
                socket()),
4            Set.of(ComponentType.CPU, ComponentType.MOTHERBOARD)
5    );
```

```
6
7       public static BuildSpecification
            gpuLengthFitsCaseSpecification = new
            PredicateSpecification(
8               b -> b.getComponents(ComponentType.GPU, Gpu.class).
                    stream().allMatch(g -> g.lenghtMm() <= b.
                    getSingleComponent(ComponentType.CASE, Case.class
                    ).maxGpuLengthMm()),
9               Set.of(ComponentType.GPU, ComponentType.CASE)
10      );
11
12      ...
```

Listing 11: A simple Java class

4.1.2 Logica

Le regole sono tutte instanziate dentro il regitrso `RulesRegistry`, la quale struttra e funzionamento verrà spiegato nella sezione riguardanti le strutture dati.
Le regole vengono verificate da `CompatibilityChecker` con i metodi `evaluateAll()` e `reevaluate()`.

Se ogni singola regola venisse valutata senza nessun meccanismo di guardia si avrebbero che messaggi di violazioni per stati nel quale la build non è, per esempio: non si può avere un messaggio "Il socket della CPU non corrisponde con quello della MOBO" se una delle due è mancante. Inoltre, sia per il meccanismo di placehoder dei messaggi e di alcuni metodi usati come `build.getSingleComponent` (che può lanciare un'eccezione) producono degli errori se eseguiti senza un controllo.

Per esempio, se si volesse controllare se il socket corrisponde tra CPU e scheda madre oppure quando una o entrambe le due sono mancanti, si sta cercando di effettuare delle operazioni con degli oggetti in uno stato non corretto (in quando non si sono verificate le precondizioni): si ha infatti `IllegalStateException`.
Stessa cosa accade anche nel secondo esempio, sempre per la mancanza di una cpu nella build.

Come meccanismo di guardia per evitare le problematiche menzionate, è stato aggiunto `implies()` agli operatori logici di Specification:

$$A \to B \quad \iff \quad \neg A \vee B$$

L'aspetto fondamentale da considerare quando si parla di operatori logici in linguaggi di programmazione è quello dello *short-circuit*, ovvero corto circuito:

> La valutazione a corto circuito, valutazione minima o valutazione di McCarthy è un meccanismo relativo agli operatori booleani binari per cui il secondo

`new BuildRule(`
`  14,`
`  "SOCKET-MISMATCH",`
`  Specs.socketMatchSpecification,`
`  Severity.ERROR,`
`  "CPU {CPU} and Motherboard {MOTHERBOARD} socket must match"`
`),`
	`Exception in thread "main" java.lang.IllegalStateException: No component found for type: CPU`

`new BuildRule(`
`  13,`
`  "MISSING-COOLER",`
`  Specs.cpuHasBundledCoolerSpecification`
`  .or(Specs.atLeast(ComponentType.COOLING, 1),`
`  Severity.ERROR,`
`  "CPU {CPU} ships without a cooler and none was selected"`
`),`
	`Exception in thread "main" java.lang.IllegalStateException: No component found for type: CPU`

Tabella 2: Esempi di regole ed eccezioni corrispondenti.

operando viene valutato unicamente se il valore del primo operando non è sufficiente da solo a determinare il risultato dell'espressione: quando il primo argomento dell'operatore logico AND è falso, il valore dell'intera espressione dovrà essere necessariamente falso; e quando il primo argomento dell'operatore logico OR è vero, il valore dell'intera espressione dovrà essere necessariamente vero. wikipedia da citare

Appurato questo aspetto fondamentale, si può adesso entrare nei dettagli di come l'implicazione fa da guardia. Se si prendono due variabili booleane A e B e si valuta $A \to B$:

- Se A è vero, allora l'espressione diventa

$$\neg \text{True} \vee B \quad \iff \quad B$$

  e dipende *unicamente* dal valore di B.

- Se A è falso, allore l'espressione diventa

$$\neg \text{False} \vee B \quad \iff \quad \text{True} \vee B$$

  ma per il cortocircuito B viene ignorata e di conseguenza non viene neanche valutata, dando direttamente come risultato True.

4.1.3 Composizione con Guardie

Una volta definita la logica dietro il meccanismo dell'implicazione utilizzando il cortocircuito, si passa all'applicazione di questa nel codice, per creare delle specifiche con guardia.

Data la specifica di guardia `GuardSpecification`, la specifica che si vuole valutare `ActualSpecification` e la loro composizione `ResultSpecification = GuardSpecification.implies(ActualSpecification)`, nel codice si hanno le seguenti conseguenze sul risultato di `ResultSpecification.isSatisfiedBy(Object)`:

- Se `GuardSpecification.isSatisfiedBy(Object) == True`, allora `ResultSpecification.isSatisfiedBy(Object)` dipende *unicamente* dall'esito di `ActualSpecification.isSatisfiedBy(Object)`
- Se `GuardSpecification.isSatisfiedBy(Object) == False`, allora per il cortocircuito logico `ResultSpecification.isSatisfiedBy(Object)` ha come esito `True`, ma sopratutto **non viene invocato** il metodo `ActualSpecification.isSatisfiedBy(Object)`.

Quindi in conclusione, quando vengono valutate delle regole che presentano una guardia, se l'esito è negativo allora si trovano in uno stato nel quale questa è valutabile, e può generare conseguentemente le relative `Violation`. Nel caso sia vera invece, o la specifica è in uno stato valutabile e risulta rispettata, oppure è in uno stato non valutabile, ma dando come risultato `True` questa valutazione è **trasparente** e non genera nessuna `Violation` (e si prevengono tutte le problematiche iniziali).

Riprendendo l'esempio precedente (quello della tabella, da aggiungere refernce), si riscrivono le regole nel seguente modo:

```
new BuildRule(
    14,
    "SOCKET-MISMATCH",
    Specs.mutipleExcatclyOne(ComponentType.CPU, ComponentType.
       MOTHERBOARD).implies(Specs.socketMatchSpecification),
    Severity.ERROR,
    "CPU {CPU} and Motherboard {MOTHERBOARD} socket must match"
),
new BuildRule(
    12,
    "MISSING-COOLER",
    Specs.excatcly(ComponentType.CPU, 1).implies(Specs.
       cpuHasBundledCoolerSpecification.or(Specs.atLeast(
       ComponentType.COOLING, 1))),
    Severity.WARNING,
    CPU {CPU} ships without a cooler and none was selected"
),
```

Listing 12: A simple Java class

4.2 Observer

L'implementazione di `Observer` è quella standard di GoF, con la sola differenza che le interfaccie sono dichiarate in modo generico e viene utilizzato un oggetto apposito per la notifica (`Event`).

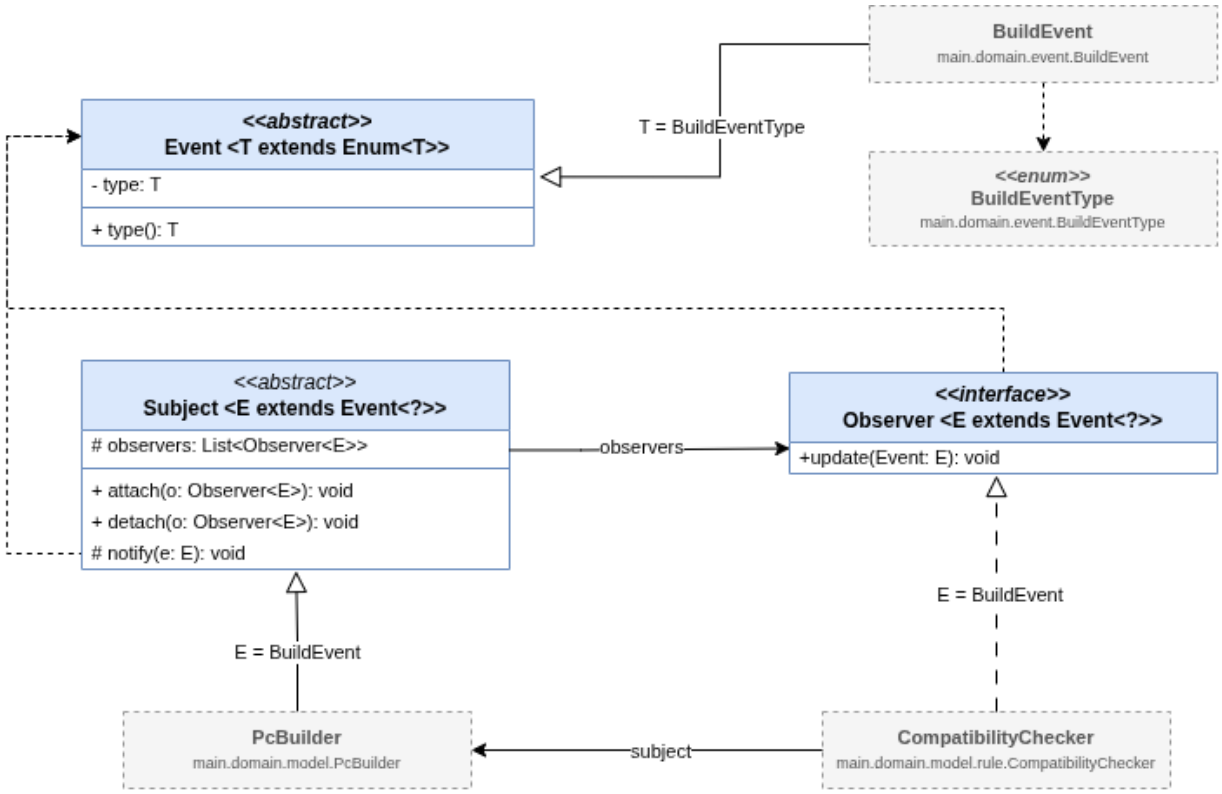


Figura 14: Enter Caption

Questa scelta di implementazione è simile a quella di *EventListener* in java, dove invece di avere solamente un *push* o un *pull*, vengono effettuati entrambi: viene effettuato un push dell'oggetto `Event` in base al quale poi gli oggetti `Observer` possono attingere ai dati di `Subject`.

Quando `notify()` viene chiamato dentro `PcBuilder` (nei metodi `addComponent()`, `removeComponent()` e `RestoreMemento()`), viene effettuato un push di un `BuildEvent`, per esempio `notify(new BuildEvent(BuildEventType.ADDED, c));`

Dopodichè l'evento viene passato a `CompatibilityChecker` (ovvero l'Observer) che col metodo `update()` provvede ad elaborare l'evento

```
1     @Override
2     public void update(BuildEvent event) {
3         if (event.type() == BuildEventType.RESTORED)
4             evaluateAll();
5         else {
6             for (BuildRule rule : registry.lookup(Set.of(event.
                  componentType()))) {
```

```
7                   reevaluate(rule);
8               }
9           }
10      }
```

Listing 13: A simple Java class

Siccome la lista delle BuildRule è molto grande, per evitare di dover ogni volta rielaborare ognuna di esse, si controlla se l'evento è di tipo `BuildEventType.ADDED o .REMOVED`. Se è questo il caso, si procede alla rivalutazione solo delle regole legate al componente che è stato aggiunto o rimosso, sfruttando la `InvertedIndexMap` di `BuildRegistry`:

```
PcBuildSession.add(c)
  -> PcBuilder.addComponent(c)
       components.get(c.type()).add(c);                    // 1. mutate state
       notify(new BuildEvent(ADDED, c));                   // 2. then notify
         -> CompatibilityChecker.update(event)
              registry.lookup(Set.of(event.componentType()))   // 3. dirty set
              -> reevaluate(rule) for each                     // 4. re-check
```

Nel caso invece che l'evento sia quello di `RESTORED`, vengono rivalutate tutte le regole.

Insieme a Specification, Observer è un altro componente fondamentale dell'applicazione, questo duo è responsabile della maggior parte della logica.

4.3 Memento

Il design pattern Memento serve a salvare gli stati interni della build, che in questo caso sono utilizzati per implementare le azioni di *undo/redo*(Ctrl+Z/Ctrl+Y). L'implementazione cerca di seguire il più possibile quella standard di GoF, ma siccome manca l'operatore *friend* del C++ si ricorre ad un altro meccanismo per garantire che lo stato registrato da uno snapshot sia accessibile solo dall'Originator: `BuildSnapshot` implementa l'interfaccia `Memento` (anche se vuota) e aggiunge il metodo `getState()`. Così facendo `SnapshotHistory` opera su l'interfaccia `Memento` e non conosce l'esistenza della classe privata interna `PcBuilder.BuildSnaphot`, ma quando l'oggetto `Memento` viene passato come argomento a `PcBuilder.restoreMemento(Memento)`, quest'utlimo può accedera allo stato nascosto tramite un cast a `BuildSnapshot`.

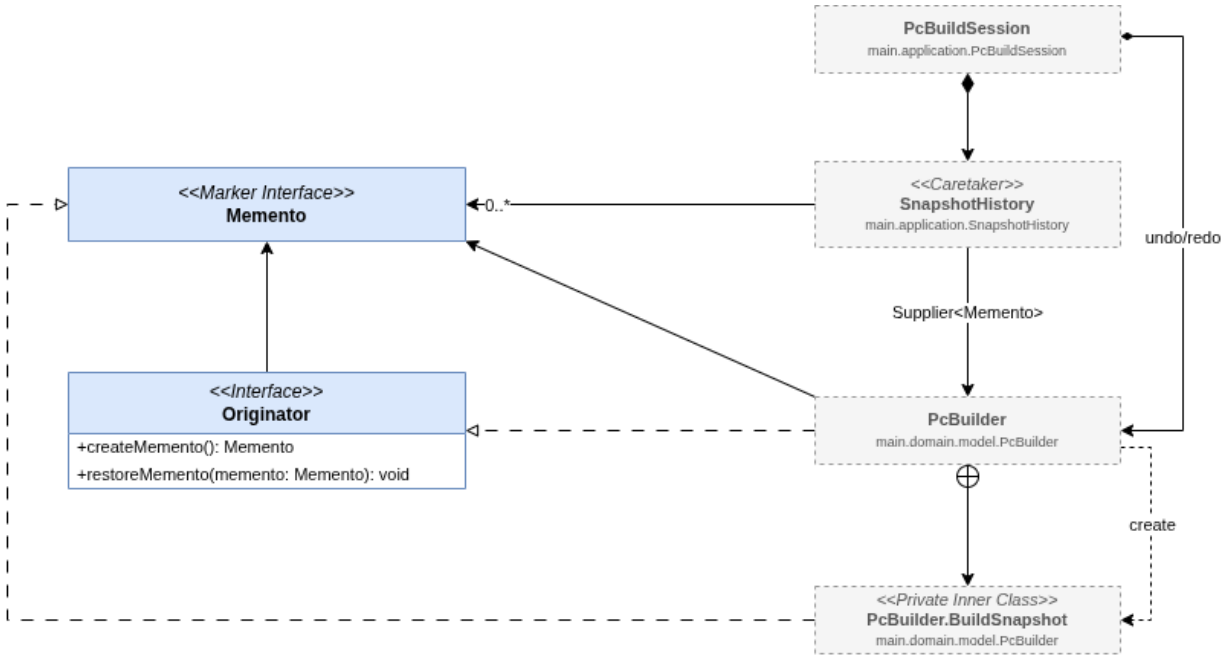


Figura 15: Enter Caption

Occore inoltre fare un confronto col design pattern `Command`...

4.4 Facade

Per quanto riguarda l'utilizzo da parte dell'utente finale, conviene semplificare tutti i meccanismi dell'applicazione in un'unica interfaccia unificata al sottosistema, che funge quindi da `Facade`. Questo fa si che l'utente non deva accedere ai singoli oggetti che compongono l'applicazione, ma si interfaccia a `PcBuildSession` che semplifica l'accesso a quest'ultimi, ma che comunque non lo priva se necessario.

I vantaggi che si ottengono sono i seguenti:

- Disaccoppiamento tra il client e il sistema, permettendo maggiore flessibilità.
- Permette di effettuare controlli aggiuntivi sulle azioni degli utenti e preserva le invarianti (come per esempio quella di eseguire `save()` ad ogni cambiamento).
- Restituisce solo una `BuildView` della build come risultato.
- Crea un livello di astrazione *Application Layer*.

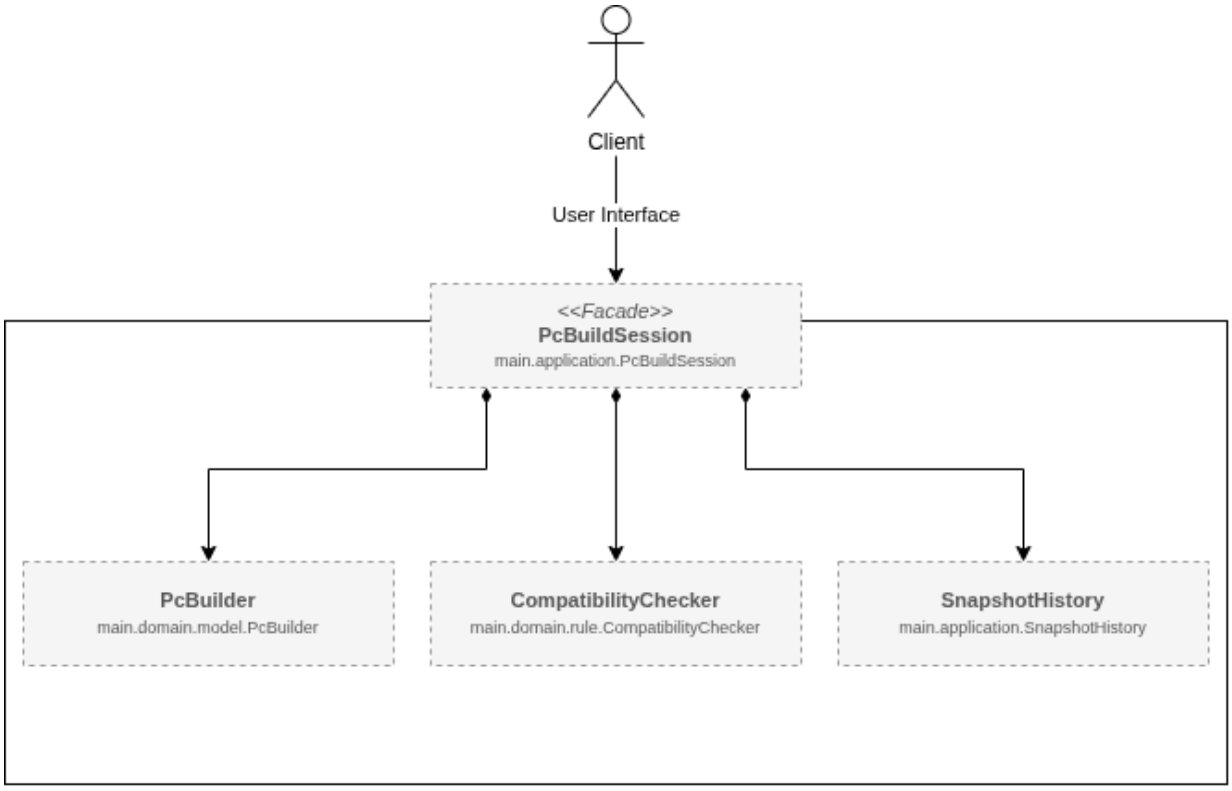


Figura 16: Enter Caption

4.5 Altri Design Pattern

- **Factory**
- **Proxy**
- **Interpreter**

4.6 Strutture Dati Utilizzate

La struttura dati principale, che rappresenta la build del pc, è una `EnumMap<>`, che ha come chiavi i tipi di componenti, e come valori le liste di quei componenti.

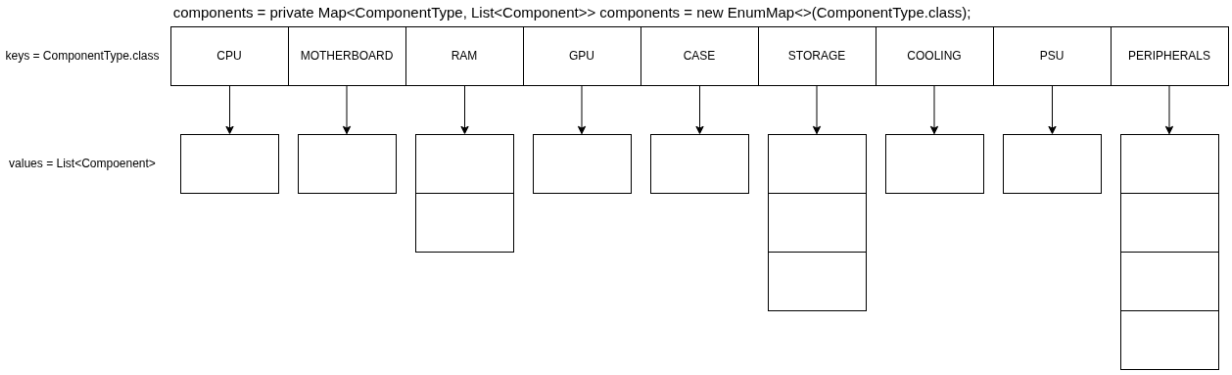


Figura 17: Enter Caption

Per quanto riguarda invece le strutture dati di supporto alle regole, si hanno due strutture dati. La prima `LinkedHasehdMap<>`, indicizzata tramite il nome (univoco) della regole e contiene la regola stessa. Questa è la struttura "base" delle regole, e la mappa serve per potere accederci direttamente tramite il loro nome per le varie operazioni che si possono effettuare su di esse (aggiunta e rimozione).

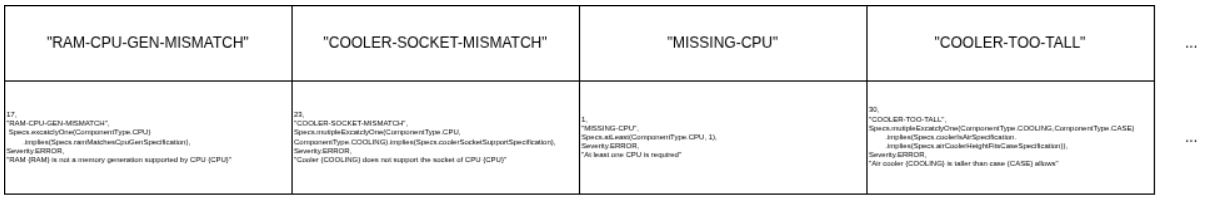


Figura 18: Enter Caption

La seconda invece inverte la mappa, prendendo un altra chiave ovvero i componenti presenti in `BuildRule.involvedComponents`. La struttura è definita in `InvertedIndexMap`

```java
//InvertedIndexMap.java
package main.domain.rule;


public abstract class InvertedIndexMap<K extends Enum<K>, V> {

    protected final Map<K, List<V>> index;
    private final Function<V, Set<K>> keysOf;

    public InvertedIndexMap(Class<K> keyType, Function<V, Set<K>>
        keysOf) {
        this.index = new EnumMap<>(keyType);
        this.keysOf = keysOf;
        for(K key : keyType.getEnumConstants()) {
            index.put(key, new ArrayList<>());
        }
    }

    public void add(V value) {
        Set<K> keys = keysOf.apply(value);
        for (K key : keys)
            index.get(key).add(value);
    }

    public boolean remove(V value) {
        boolean removed = false;
        Set<K> keys = keysOf.apply(value);
        for (K key : keys) {
            // Keys are pre-initialized in the constructor and never
               removed, so the list is always present.
```

```
29              removed |= index.get(key).remove(value);
30          }
31          return removed;
32      }
33
34      public Set<V> lookup(Set<K> keys) {
35          // VERY Important that this is a set, we don't want repeting
                values coming from different keys
36          Set<V> result = new LinkedHashSet<>();
37          keys.forEach(k -> result.addAll(index.getOrDefault(k, List.of
                ())));
38          return result;
39      }
40
41  }
```

Listing 14: A simple Java class

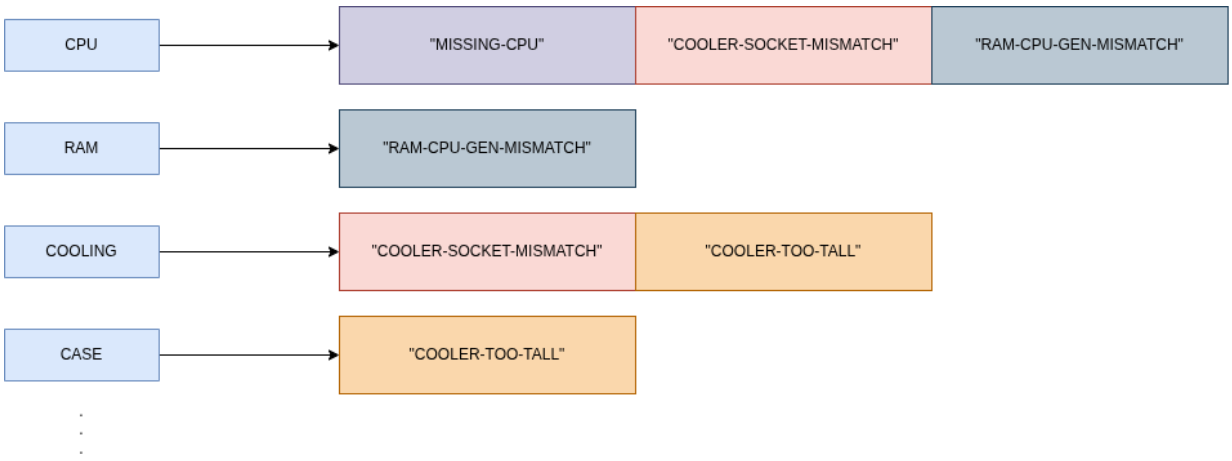


Figura 19: Enter Caption

Quindi la mappa si basa sempre su una `EnumMap<>` come `components`, ma i valori sono tutte le regole che tra i componenti coinvolti contengono anche quello dell'indice. Per questa definizione ci saranno quindi dei duplicati nei valori, ovvero le regole che dipendono da due o più componenti.

Quando si va ad aggiungere/rimuovere/rimpiazzare un componente, vengono prese in considerazioni **solo** le regole che comprendono quella tipologia di componente (...), in modo da non dover rivalutare tutta la lista di regole. provsporvvsvsvvs jjjj jjjjbj